**Programare Web / Dezvoltarea Aplicațiilor Web**

Tehnologia Informației, anul IV · Informatică, anul II · Anul universitar 2026–2027, semestrul I

LABORATORUL 08

# API REST complet (CRUD) pentru recenzii

*Controller-ul ReviewController: citim, adăugăm, modificăm și ștergem recenzii prin HTTP și JSON.*

| | |
|---|---|
| **Tema** | Temele 9 și 11 din planul disciplinei (examene.pe.ub.ro): persistența datelor și servicii web (API REST) |
| **Durata** | 2 ore |
| **Aveți nevoie de** | entitatea `Review` din laboratorul 07, Swagger și Postman |
| **La final aveți** | `ReviewController.php` cu șase rute testate și un commit pe branch-ul vostru |

## 1. Obiective

La finalul laboratorului veți putea:

- explica ce este o resursă REST, rolul metodelor HTTP și al codurilor de stare (200, 201, 204, 400, 404);
- scrie un controller Symfony care citește JSON, validează datele și salvează cu Doctrine;
- testa un API în Swagger și în Postman, inclusiv cu date greșite.

## 2. Pe scurt: REST

**REST** (*Representational State Transfer*) este un stil de proiectare a serviciilor web [2]. Datele sunt organizate în **resurse**, fiecare cu o adresă (URL): `/api/reviews` este colecția de recenzii, `/api/reviews/7` este recenzia cu id-ul 7. Clientul (browserul, React, Postman) spune **ce vrea să facă** prin metoda HTTP, iar serverul răspunde cu JSON și cu un **cod de stare**: **2xx** succes, **4xx** greșeala clientului (date invalide, resursă inexistentă), **5xx** eroare în server [1], [4]. La date greșite, un API bun nu „cade” cu 500, ci răspunde **400** cu un mesaj clar.

| Metoda și ruta | Ce face | Răspuns la succes |
|---|---|---|
| `GET /api/reviews` | listează toate recenziile | 200 OK |
| `GET /api/reviews/{id}` | o singură recenzie (404 dacă lipsește) | 200 OK |
| `GET /api/products/{id}/reviews` | recenziile unui produs | 200 OK |
| `POST /api/reviews` | creează o recenzie (400 la date greșite) | 201 Created |
| `PUT /api/reviews/{id}` | înlocuiește toate câmpurile recenziei | 200 OK |
| `PATCH /api/reviews/{id}` | modifică doar câmpurile trimise (exercițiu ★) | 200 OK |
| `DELETE /api/reviews/{id}` | șterge recenzia | 204 No Content |

## 3. Pornirea

1. Porniți **Docker Desktop** (mesajul **Engine running**). Comenzile se scriu în terminalul editorului, în folderul `daw-template-ti-info-2026-2027`; pentru probleme, vedeți metodologia [5].
2. Verificați pe ce branch sunteți: `git branch` marchează ramura curentă cu `*` (sau `git status`, pe primul rând: `On branch …`). Doar dacă nu sunteți deja pe branch-ul vostru, treceți pe el; apoi porniți containerele (înlocuiți `popescu.ion`):

```
git branch                # ramura curenta are * in fata
# doar daca NU sunteti deja pe branch-ul vostru:
git checkout popescu.ion   # sau: git switch popescu.ion
docker compose -f backend/compose.yaml up -d
docker compose -f frontend/compose.yaml up -d
```

## 4. Controller-ul ReviewController

Controller-ul primește cererea HTTP, citește cu **repository-urile**, scrie cu **EntityManagerInterface** și răspunde cu `$this->json(...)`, după modelul lui `ProductController` [3]. Creați fișierul `backend/app/src/Controller/ReviewController.php` (click dreapta pe folderul `Controller` → **New File**) și scrieți codul de mai jos, pe care îl citim pe bucăți.

### a) Antetul, dependențele și un exemplu de corp JSON pentru Swagger

```
<?php

namespace App\Controller;

use App\Entity\Review;
use App\Repository\ProductRepository;
use App\Repository\ReviewRepository;
use Doctrine\ORM\EntityManagerInterface;
use OpenApi\Attributes as OA;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Exception\JsonException;
use Symfony\Component\HttpFoundation\JsonResponse;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Attribute\Route;

#[OA\Tag(name: 'Reviews')]
final class ReviewController extends AbstractController
{
  private const EXAMPLE = ['author' => 'Ana Popescu', 'rating' => 5,
    'comment' => 'Recomand!', 'productId' => 1];

  public function __construct(
    private EntityManagerInterface $em,
    private ReviewRepository $reviews,
    private ProductRepository $products,
  ) {
  }
```

### b) Citirea: trei rute GET

```
  #[Route('/api/reviews', methods: ['GET'])]
  public function list(): JsonResponse
  {
    $items = $this->reviews->findBy([], ['createdAt' => 'DESC']);
    return $this->json(array_map(fn (Review $r) => $r->toArray(), $items));
  }
```

```
#[Route('/api/reviews/{id<\d+>}', methods: ['GET'])] // {id<\d+>}: doar cifre
public function show(int $id): JsonResponse
{
  $review = $this->reviews->find($id);
  if (!$review) {
    return $this->json(['error' => 'Recenzia nu există.'], 404);
  }
  return $this->json($review->toArray());
}

#[Route('/api/products/{id<\d+>}/reviews', methods: ['GET'])]
public function byProduct(int $id): JsonResponse
{
  $product = $this->products->find($id);
  if (!$product) {
    return $this->json(['error' => 'Produsul nu există.'], 404);
  }
  $items = $this->reviews->findBy(['product' => $product], ['createdAt' => 'DESC']);
  return $this->json(array_map(fn (Review $r) => $r->toArray(), $items));
}
```

### c) Scrierea: POST, PUT și DELETE

```
#[Route('/api/reviews', methods: ['POST'])]
#[OA\RequestBody(content: new OA\JsonContent(example: self::EXAMPLE))]
public function create(Request $request): JsonResponse
{
  $review = new Review();
  $errors = $this->fill($review, $request);
  if ($errors) {
    return $this->json(['errors' => $errors], 400);
  }
  $this->em->persist($review); // obiect nou, de salvat
  $this->em->flush();          // trimite INSERT la PostgreSQL
  return $this->json($review->toArray(), 201);
}

#[Route('/api/reviews/{id<\d+>}', methods: ['PUT'])]
#[OA\RequestBody(content: new OA\JsonContent(example: self::EXAMPLE))]
public function update(int $id, Request $request): JsonResponse
{
  $review = $this->reviews->find($id);
  if (!$review) {
    return $this->json(['error' => 'Recenzia nu există.'], 404);
  }
  $errors = $this->fill($review, $request);
  if ($errors) {
    return $this->json(['errors' => $errors], 400);
  }
  $this->em->flush();
  return $this->json($review->toArray());
}
```

```
  #[Route('/api/reviews/{id<\d+>}', methods: ['DELETE'])]
  public function delete(int $id): Response
  {
    $review = $this->reviews->find($id);
    if (!$review) {
      return $this->json(['error' => 'Recenzia nu există.'], 404);
    }
    $this->em->remove($review);
    $this->em->flush();
    return new Response(null, 204);
  }
```

**d) Validarea manuală, comună pentru POST și PUT**

```
  // Verifică JSON-ul și completează recenzia; întoarce erorile (câmp => mesaj).
  private function fill(Review $review, Request $request): array
  {
    try {
      $data = $request->toArray();
    } catch (JsonException) {
      return ['body' => 'Corpul cererii trebuie să fie un obiect JSON valid.'];
    }

    $errors = [];
    $author = is_string($data['author'] ?? null) ? trim($data['author']) : '';
    if ($author === '' || mb_strlen($author) > 100) {
      $errors['author'] = 'Autorul este obligatoriu (cel mult 100 de caractere).';
    }
    $rating = $data['rating'] ?? null;
    if (!is_int($rating) || $rating < 1 || $rating > 5) {
      $errors['rating'] = 'Nota trebuie să fie un număr întreg între 1 și 5.';
    }
    $product = $this->products->find((int) ($data['productId'] ?? 0));
    if (!$product) {
      $errors['productId'] = 'Produsul indicat nu există.';
    }
    if ($errors) {
      return $errors;
    }

    $comment = is_string($data['comment'] ?? null) ? trim($data['comment']) : '';
    $review->setAuthor($author)
      ->setRating($rating)
      ->setComment($comment === '' ? null : $comment)
      ->setProduct($product);
    return [];
  }
}
```

## 5. Testarea în Swagger și în Postman

### 5.1 Swagger

1. Deschideți http://localhost:8080/api/doc, secțiunea **Reviews**. La `GET /api/reviews` → **Try it out** → **Execute** vedeți codul 200 și recenziile din laboratorul 07.
2. `POST /api/reviews` → **Try it out**: corpul vine completat din constanta `EXAMPLE`. Puneți la `productId` un id existent și apăsați **Execute**. Răspunsul are codul **201** și conține recenzia nouă, cu `id` și `createdAt` completate de server.

### 5.2 Postman: cazurile de eroare

1. În Postman, importați (**Import**) colecția `postman/DAW-API.postman_collection.json` din repository. Adăugați o cerere `POST` la `http://localhost:8080/api/reviews`, cu **Body → raw → JSON**, și trimiteți (**Send**) pe rând corpurile din tabel.

| Corpul trimis (Body → raw → JSON) | Răspuns așteptat |
|---|---|
| `{"author": "Ion", "rating": 4, "productId": 1}` | **201**, recenzia creată |
| `{"author": "", "rating": 4, "productId": 1}` | **400**, eroare la `author` |
| `{"author": "Ion", "rating": "4", "productId": 1}` | **400**: `"4"` este text, nu număr |
| `{"author": "Ion", "rating": 4, "productId": 9999}` | **400**, eroare la `productId` |
| `{"author": "Ion", "rating": 4,` (JSON incomplet) | **400**, eroare la `body` |

1. Pe un id existent (de ex. `/api/reviews/1`) trimiteți un `PUT` cu corp complet, apoi un `DELETE` și un `GET`. Salvați cererile în colecție (**Ctrl + S**).

> **Rezultat.** PUT → **200** cu datele noi; DELETE → **204** fără conținut; GET după ștergere → **404**.

## 6. Salvarea lucrului și oprirea

```
git add .
git commit -m "Laboratorul 08: API REST pentru recenzii (ReviewController)"
git push origin popescu.ion
# sau doar: git push (dupa ce ati facut o data git push -u)
docker compose -f backend/compose.yaml stop
docker compose -f frontend/compose.yaml stop
```

> **Push.** `git push origin popescu.ion` este forma completă: trimite branch-ul vostru la `origin`, adresa proiectului de pe GitHub. Forma scurtă, `git push`, merge doar dacă ați făcut anterior, o dată, `git push -u origin popescu.ion`; `-u` leagă branch-ul local de cel de pe GitHub.

## 7. Exerciții

1. Cereți în browser http://localhost:8080/api/reviews/abc. Ce cod primiți și de ce nu ajunge cererea în metoda `show()`?
2. Adăugați ruta `GET /api/products/{id}/rating`, care întoarce `{"productId": 1, "count": 3, "average": 4.33}`. Indiciu: calculați media în PHP, din lista primită de la `findBy()`.
3. ★ (opțional) Implementați `PATCH /api/reviews/{id}`, care modifică **doar** câmpurile prezente în JSON. Ce diferență observați față de PUT când trimiteți doar `{"rating": 3}`?

## 8. Ce arătați la final

- în Swagger, `GET /api/products/{id}/reviews`; în Postman, un `POST` reușit (201), două cu erori (400) și un `DELETE` (204);
- commit-ul „Laboratorul 08” pe GitHub, pe branch-ul vostru, cu fișierul `ReviewController.php`.

## Resurse pentru exersare

- **W3Schools – PHP (cu exemple interactive):** https://www.w3schools.com/php/
- **Tutorial PHP în limba română – introducere (video):** https://www.youtube.com/watch?v=JaTO46oiCEM
- **Postman – tutorial video (în engleză):** https://www.youtube.com/watch?v=VywxIQ2ZXw4
- **Symfony – Controller:** https://symfony.com/doc/current/controller.html
- **MDN – metodele HTTP:** https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods

## Bibliografie

**[1]** E. Pruteanu și I. Furdu, *Modele, tehnologii, servicii și aplicații integrate de tip R.I.A. pentru web*. Bacău: Alma Mater, 2022.

**[2]** R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, teză de doctorat, University of California, Irvine, 2000.

**[3]** Symfony, „Controller”, documentație oficială. symfony.com/doc/current/controller.html

**[4]** MDN Web Docs, „HTTP response status codes”. developer.mozilla.org

**[5]** C.-M. Papară, *Metodologie de lucru și template de proiect – Programare Web / Dezvoltarea Aplicațiilor Web*. Universitatea „Vasile Alecsandri” din Bacău, 2026.